Design Rationale for REQ1

The UML and Sequence Diagrams for REQ1 can be found in the same directory.

Functionality

The Teleportable interface defines the contract for any class implementing it to require the method teleport(Actor actor, GameMap map).

The TeleportAction class that extends Action allows items such as THESEUS to be used as a teleportation device for an actor to be teleported to a certain location.

The Theseus class that extends Item and implements Purchasable and Teleportable, can be purchased for $100 at the terminal and dropped on the floor to allow an actor standing on it to be teleported to a random location of the current map they are in.

The PlayableMap class that extends GameMap stores a list of the playable maps along with their layouts and names. This allows the maps to be uniquely identified.

The Terminal class that extends Ground has been given new functionality. It now allows a player to travel to any map other than the one they are currently on.

Object-Oriented Design Principles

Every class only contains their respective capabilities. This means that they are responsible for only one part of the program's functionality. (Single Responsibility Principle)

Pros:

- Bugs will be easy to find as each class deals with a specific function of the system.

Cons:

- There may be an abundance of classes which could make maintaining the system difficult.

The Teleportable interface allows future teleportable items to simply implement the interface without altering existing code. This keeps it open for extension but closed for modification. (Open-closed Principle)

Pros:

- The system is more extensible.

Cons:

- If the interface requires modification, every file implementing the interface will have to be refactored.

Since the child classes extend their respective parent classes (e.g. Theseus extends Item), which define common behaviour of all their subclasses, the subclasses can be used in place of their respective parent classes without disrupting the behaviour of the program. (Liskov Substitution Principle)

Pros:

- The system is more flexible and achieves polymorphism.

Cons:

- Subclasses overriding parent classes unnecessarily could cause the performance to suffer.

As THESEUS is currently the only teleportable item and makes full use of the method in the Teleportable interface, THESEUS is not implementing methods that it does not need. (Interface Segregation Principle)

Pros:

- The teleportable item only contains relevant methods which will avoid unnecessary implementations.

Cons:

- The use of interfaces could result in an excessive amount of interfaces which could make it difficult to maintain the system.

The concrete child classes do not depend on another concrete class but on the abstract parent class instead (e.g. Theseus class extends Item abstract class). (Dependency Inversion Principle)

Pros:

- Loose coupling allows for easier modification which makes the system more flexible.

Cons:

- Too many abstract classes could overcomplicate the system which may make it difficult to maintain the system.

Justification

Despite the potential increased complexity, the SOLID principles allow for easier modification and extensibility.